



Tecnológico de Monterrey

Reporte: RISC-V Pipeline

Rodrigo Armando Reveles Picie A01641220

Mariana Eileen Martínez Mancilla A01640695

Ricardo Sebastián González Rivas A01646105

Marco Natsumi Rabiela Mun A01647377

Omar Verduzco Velazquez A07019516

14 de junio de 2026

Diseño de sistemas en chip (Gpo 501)

I. Descripción del funcionamiento del procesador

El procesador *pipeline* es una versión del *single-cycle*, tomando en cuenta los procesos más largos del sistema. En otras palabras, todos los protocolos más tardados, leyendo y escribiendo la memoria y el *register file* y usando la ALU, se hacen parte de una etapa individual en *pipelining*. La ventaja está dada por la capacidad de completar varias tareas simultáneamente (Roberts, n.d.). Es decir, cada vez que una tarea es liberada por una etapa, otra tarea se puede introducir a esa misma etapa, mientras que la tarea anterior avanza al siguiente proceso de pipeline.

En cuanto al funcionamiento físico, el procesador pipeline utiliza varios registros, con una entrada de reloj, que guarda los resultados de la etapa pasada. Al completarse un ciclo de reloj, los datos guardados pasan a la siguiente etapa y de nuevo se guardan en otro registro, hasta completarse la tarea entera.

Además, como cada etapa representa una fracción del sistema completo, cada proceso se completa notablemente más rápido. Los procesadores pipeline llegan a tener una velocidad de reloj más rápida que los procesadores single-cycle y sacan un mejor rendimiento en general (Harris & Harris, 2022, p. 461).

II. Explicación del pipeline de 5 etapas

Un procesador pipeline está dividido en 5 etapas: fetch, decode, execute, memory y writeback. Cada proceso representa acceso, escritura o alguna acción sobre los elementos más tardíos de un procesador RISC-V. Funcionan de la siguiente manera:

Fetch: En la etapa de *fetch*, el procesador lee las instrucciones de la memoria de instrucciones. Aquí también se pasa la dirección de la siguiente instrucción *PCNext*.

Decode: En la etapa de decodificación, se extrae cada operando de la instrucción completa del *register file* y se obtienen las señales de control. Es en esta etapa donde se encuentra la unidad de control.

Execute: En la etapa de ejecución, se completan las operaciones con la ALU principal y con la ALU para cálculos de dirección de la memoria de instrucciones.

Memory: En la etapa de memoria, el procesador hace alguna lectura o escritura en la memoria, si es necesario.

Writeback: Finalmente, en la etapa de *writeback*, el procesador escribe el resultado en algún registro del *register file*, si es necesario.

Es importante destacar que no todas las instrucciones hacen uso de todas las etapas. Por ejemplo, la instrucción *sw* no hace uso de *writeback*; su última etapa activa es la de memoria.

III. Funcionamiento de la unidad de Hazard

Un *hazard* ocurre cuando una instrucción depende del resultado de otra instrucción que todavía no se ha completado.

En nuestro módulo *Hazard_Unit* gestionamos los riesgos a través de los mecanismos de *forwarding*, *stall* y *flush*. Para resolver los riesgos de datos con instrucciones de tipo registro (Tipo R), utilizamos una lógica de anticipación que compara los registros fuente *Rs1E* y *Rs2E* con los registros destino en las etapas de memoria (*RdM*) y de escritura (*RdW*). Si hay una coincidencia y se habilita la escritura (*RegWriteM* o *RegWriteW*), las señales *ForwardAE* y *ForwardBE* desvían el dato actualizado directamente hacia la ALU, ignorando siempre el registro cero.

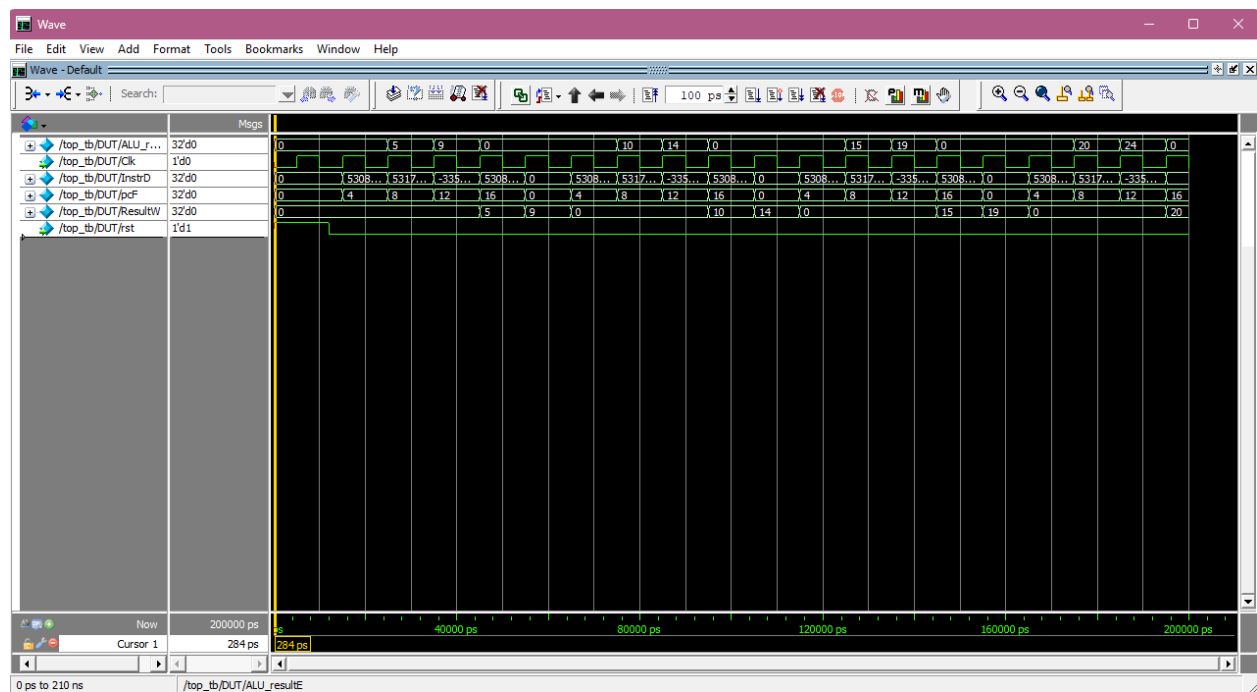
Por otro lado, en el caso de las instrucciones de carga en memoria, si la instrucción que se encuentra en la etapa de decodificación depende de ese dato (*Rs1D* o *Rs2D* coinciden con *RdE*), el *forwarding* no es suficiente. Por ello, activamos la señal *lwStall*, la cual bloquea las etapas de

Fetch y Decode y, simultáneamente, genera un *FlushE* para inyectar una burbuja en la etapa de ejecución.

Finalmente, para los riesgos de control, si se detecta un salto tomado en la etapa de Execute, la señal *PCSrcE* se activa y el módulo limpia las instrucciones incorrectas que entraron al pipeline aplicando un flush a las etapas de Decode y Execute.

IV. Resultados y capturas de simulación

Captura del Top, las demás capturas estan en un README.md dentro del repositorio.



V. Conclusiones

Tras realizar el desarrollo e implementación de un *pipeline* de cinco etapas, se concluye que es una solución altamente eficiente frente a las limitaciones de los diseños de *Single-Cycle*. Al separar el camino de datos en las etapas independientes de *Fetch*, *Decode*, *Execute*, *Memory* y *Writeback*, es posible reducir el periodo del reloj y maximizar el rendimiento general del sistema a través del paralelismo a nivel de instrucciones. No obstante, estas mejoras introducen una mayor complejidad debido a la superposición de tareas en el tiempo, lo que da origen a los riesgos o *hazards*.

Es por ello que la *Hazard Unit* resulta fundamental en el diseño, ya que actúa como el sistema de control que preserva la integridad del flujo de ejecución en todo el procesador.

VI. Referencias

Harris, S. L., & Harris, D. M. (2022). *Digital design and computer architecture: RISC-V edition*. Morgan Kaufmann.

Roberts, E. (n.d.). *Pipelining*. Stanford University.
<https://cs.stanford.edu/people/eroberts/courses/soco/projects/risc/pipelining/index.html>